

Conectores y protocolos de acceso a bases de datos

2. Conectores y protocolos de acceso a bases de datos

Conectores y protocolos de acceso a bases de datos

Conoces SQL y bases de datos relacionales desde 1º de DAM — pero hasta ahora las has usado siempre desde una herramienta externa (un cliente SQL, un script). En el apartado anterior has visto que el repository es la capa que habla con la base de datos, sin entrar en cómo lo hace por dentro. Este apartado responde a esa pregunta: ¿cómo habla tu programa Java, con sus clases y objetos, con una base de datos que solo entiende tablas y filas?

El problema de partida: objetos contra tablas

Imagina una clase Editorial con una lista de Libro — la misma pareja de clases que vas a usar durante todo este tema:

```
class Editorial {  
    String nombre;  
    List<Libro> libros;  
}
```

Y, en la base de datos, dos tablas relacionadas por clave foránea:

```
CREATE TABLE editorial (id INT PRIMARY KEY, nombre VARCHAR(100));  
CREATE TABLE libro (id INT PRIMARY KEY, editorial_id INT REFERENCES editorial(id));
```

Aunque representan la misma idea, la forma de trabajar con cada una es distinta:

El problema de partida (cont.)

- Tipos que no coinciden: Java tiene `List<Libro>`, `LocalDate`, `boolean`; SQL tiene `INT`, `VARCHAR`, `DATE`. No hay una correspondencia 1:1 automática.
- Relaciones expresadas de forma distinta: en Java, una editorial contiene su lista de libros (una referencia en memoria); en SQL, esa misma relación es una clave foránea en la tabla libro — hace falta un `JOIN` para reconstruirla.
- Identidad: en Java, dos objetos son iguales si `==` los considera el mismo (o si `equals()` lo dice); en la base de datos, la identidad de una fila es su clave primaria.

Esta diferencia de fondo tiene nombre: desfase objeto-relacional (impedance mismatch). No es un error de diseño de nadie — los dos mundos, orientado a objetos y relacional, modelan la información de forma distinta por naturaleza.

La solución: una herramienta ORM

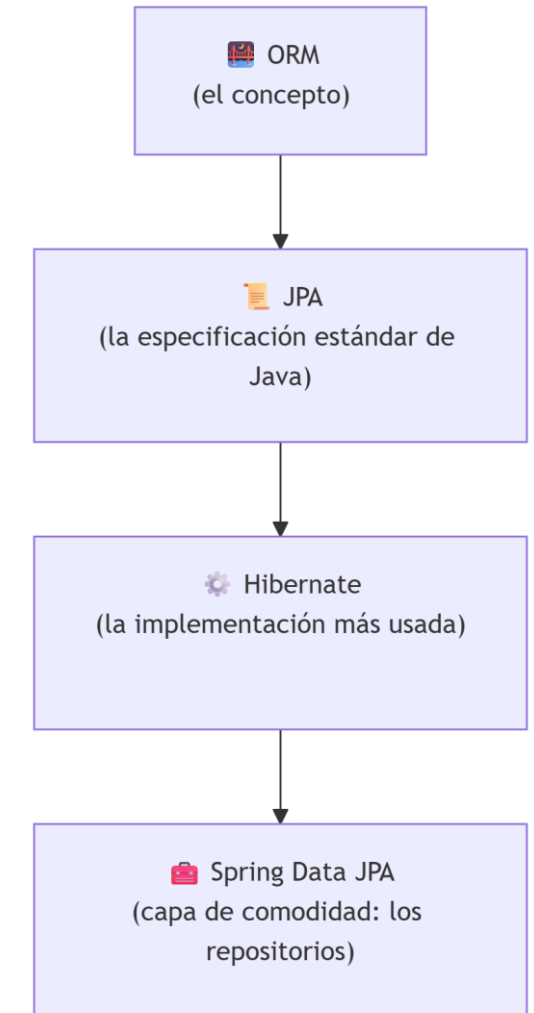
Vas a resolver este desfase con una herramienta ORM (Object-Relational Mapping): declaras, una sola vez, las reglas de correspondencia entre una clase y una tabla — y a partir de ahí, guardar, actualizar y consultar objetos se traduce automáticamente al SQL correspondiente, sin que lo escribas tú.

JPA (Jakarta Persistence API) es la especificación estándar de Java: el documento de reglas que dice qué anotaciones (`@Entity`, `@Id`, `@OneToMany`...) debe ofrecer cualquier ORM que quiera usarse en Java — sin ejecutar nada por sí sola.

Hibernate es quien cumple esas reglas de verdad: la implementación de JPA más usada, el motor que traduce cada anotación en el SQL real que llega a PostgreSQL.

La solución: una herramienta ORM (cont.)

Nombre	Qué es	Su equivalente en POO
JPA	El contrato: qué debe ofrecer un ORM en Java	Una interfaz
Hibernate	Quien lo cumple: el motor que hace el trabajo real	Una clase que la implementa



La solución: una herramienta ORM (cont.)

Spring Data JPA (el `JpaRepository` que vas a usar durante todo este tema) no es un ORM distinto — es una capa adicional de comodidad por encima de JPA/Hibernate: te genera automáticamente implementaciones de repositorios (`save`, `findById`, `findAll...`).

Hibernate no es la única opción: es la implementación de JPA más usada, pero no la única — EclipseLink es otra implementación completa de la misma especificación. Fuera de Java, cada lenguaje tiene su propio ORM (SQLAlchemy en Python, Entity Framework en C#).

Las anotaciones que vas a ver más abajo (`@Entity`, `@Id`, `@OneToMany...`) son, literalmente, cómo se declara ese mapeo — por debajo es la propia herramienta ORM en funcionamiento.

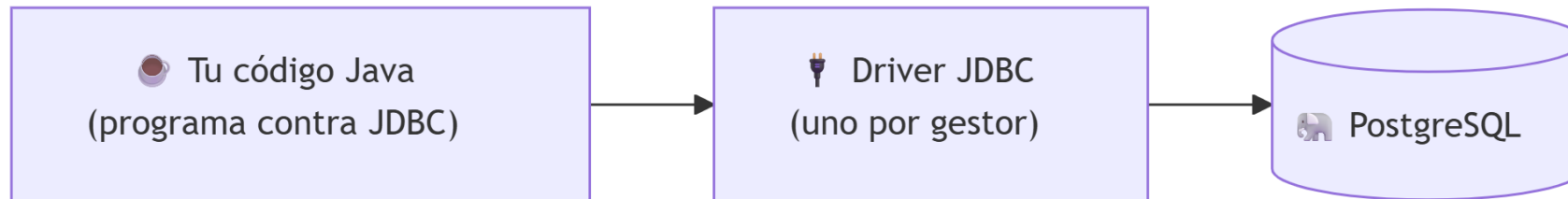
JDBC y los conectores de bases de datos

Para que tu programa Java pueda comunicarse con una base de datos necesita un conector —también llamado driver— preparado para ese gestor concreto. PostgreSQL tiene su propio driver, MySQL tiene otro y SQL Server tiene el suyo.

Sin embargo, no tendría sentido aprender una forma completamente distinta de programar para cada base de datos. Java resuelve este problema mediante JDBC (Java Database Connectivity): una API estándar que define una forma común de acceder a bases de datos relacionales desde una aplicación Java.

JDBC y los conectores (cont.)

Tu aplicación utiliza JDBC, y el driver concreto se encarga de traducir esas operaciones para que las entienda el gestor que estés usando:



La ventaja es que la forma general de programar el acceso a los datos se mantiene aunque cambie el gestor. Si una aplicación pasara de PostgreSQL a MySQL, habría que cambiar el driver y la dirección de conexión, pero no aprender una API Java completamente nueva.

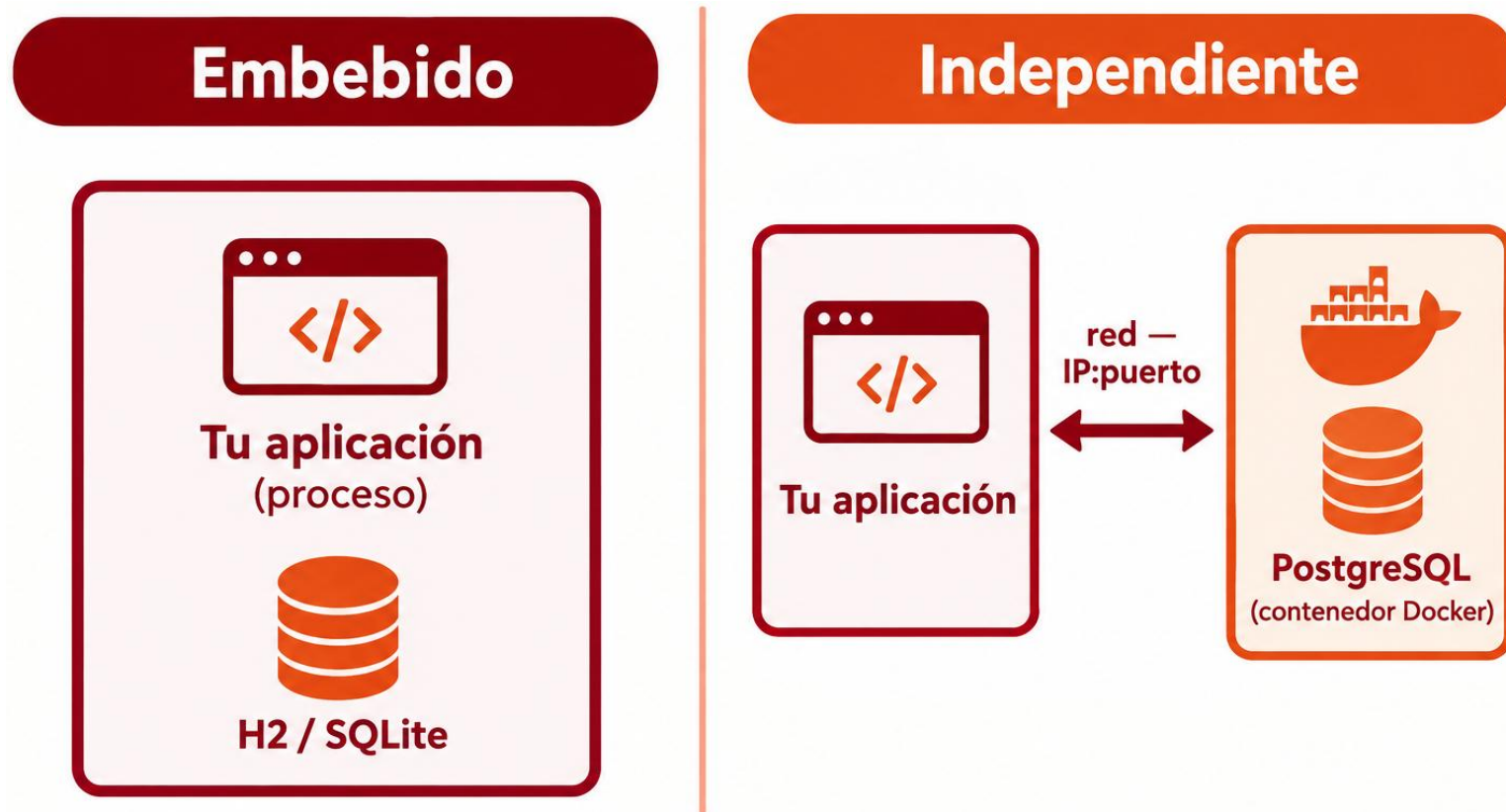
Gestor embebido vs. independiente

	Embebido	Independiente
Cómo corre	Dentro del propio proceso de tu aplicación, sin servidor aparte	Como un proceso/servidor separado, al que te conectas por red
Ejemplos	H2, SQLite	PostgreSQL, MySQL, SQL Server
Cuándo conviene	Tests rápidos, prototipos, apps de escritorio sin instalación	Aplicaciones reales, con varios clientes concurrentes

Gestor embebido vs. independiente (cont.)

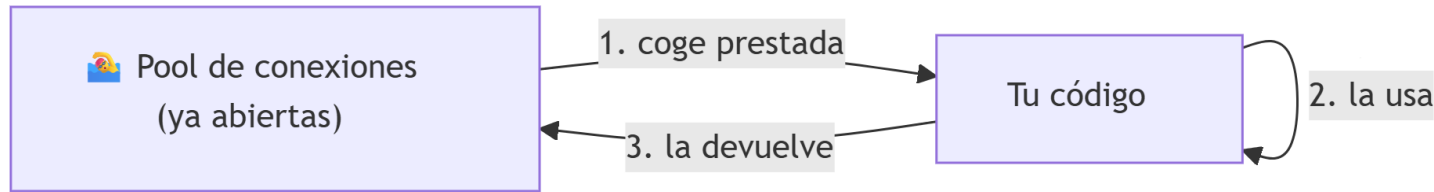
Un gestor embebido vive dentro de tu propio proceso: no hay nada que instalar ni levantar aparte, arranca y desaparece con tu aplicación. Es rápido de poner en marcha, pero normalmente no está pensado para que varias aplicaciones distintas lo usen a la vez.

Un gestor independiente corre como su propio servicio, escuchando en un puerto de red, y puede atender a muchos clientes al mismo tiempo. Es el mismo concepto de IP/puerto del apartado 1: un contenedor de PostgreSQL, separado de tu aplicación, al que te conectas por IP/puerto.



Pooling de conexiones

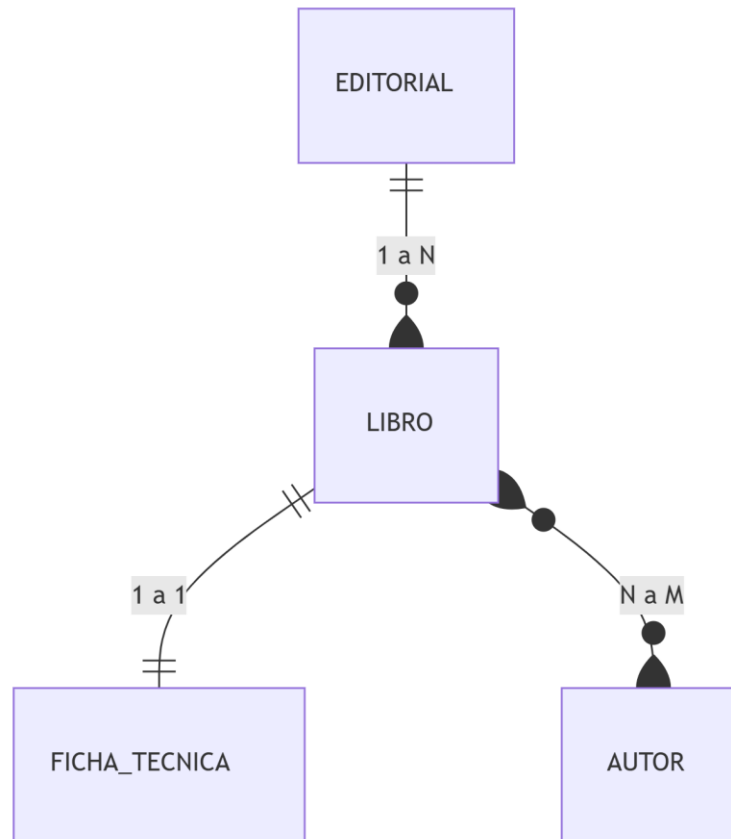
Abrir una conexión a una base de datos no es gratis: implica negociación de red, autenticación, reserva de recursos — tiene un coste real, medible en milisegundos. El pooling de conexiones resuelve esto manteniendo un conjunto de conexiones ya abiertas y listas para usar.



Analogía: taquillas ya abiertas. Forzar una cerradura nueva cada vez que necesitas guardar algo es lento. Un pool de conexiones es como un conjunto de taquillas que ya están abiertas de antemano: coges una libre, la usas, y la dejas lista para el siguiente.

Tipos de relación entre entidades

Cuando dos clases están relacionadas, no basta con decir "están relacionadas" — hace falta precisar cuántas instancias de una pueden asociarse con cuántas de la otra. A ese matiz se le llama multiplicidad (o cardinalidad), y determina qué anotación JPA usas para mapear la relación.



Tipos de relación entre entidades (cont.)

Relación	Ejemplo en la librería	Anotación
Uno a uno (1:1)	Un libro tiene una única ficha técnica con sus datos extendidos	@OneToOne en los dos lados
Uno a muchos (1:N)	Una editorial publica muchos libros	@OneToMany + @ManyToOne
Muchos a muchos (N:M)	Un libro puede tener varios autores, y viceversa	@ManyToMany en los dos lados

Uno a uno: Libro y FichaTecnica

El lado Libro — inverso de la relación (mappedBy):

```
@Entity
public class Libro {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @OneToOne(mappedBy = "libro", cascade = CascadeType.ALL)
    private FichaTecnica fichaTecnica;
}
```

Uno a uno: Libro y FichaTecnica (cont.)

El lado FichaTecnica — el que tiene la clave foránea real:

```
@Entity
public class FichaTecnica {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private Integer numeroPaginas;
    private String idioma;

    @OneToOne
    @JoinColumn(name = "libro_id", unique = true)
    private Libro libro;
}
```

Lo único que distingue un 1:1 de un 1:N en la base de datos es la columna `unique = true`: sin ella, varias fichas técnicas podrían apuntar al mismo libro.

Uno a muchos: Editorial y Libro

Es la relación que ya conoces del principio del apartado: una editorial tiene muchos libros, pero cada libro pertenece a una única editorial. Es la que vas a ver a continuación con código real.

Muchos a muchos: Libro y Autor

El lado Libro — con la tabla intermedia libro_autor:

```
@Entity
public class Libro {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToMany
    @JoinTable(
        name = "libro_autor",
        joinColumns = @JoinColumn(name = "libro_id"),
        inverseJoinColumns = @JoinColumn(name = "autor_id")
    )
    private List<Autor> autores;
}
```

Muchos a muchos: Libro y Autor (cont.)

El lado Autor — inverso de la relación:

```
@Entity
public class Autor {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nombre;

    @ManyToMany(mappedBy = "autores")
    private List<Libro> libros;
}
```

Ninguna de las dos tablas puede llevar la clave foránea: la solución es una tabla intermedia (libro_autor) con dos claves foráneas. `@JoinTable` le dice a Hibernate cómo se llama y cuáles son sus columnas.

Las piezas en un proyecto: el gestor

Un docker-compose.yml como este levanta PostgreSQL como gestor independiente:

```
services:
  postgres:
    image: postgres:18-alpine
    environment:
      POSTGRES_DB: libreria_db
      POSTGRES_USER: libreria_user
      POSTGRES_PASSWORD: password123
    ports:
      - "5432:5432"
```

Una mejora posible: H2 embebido para tests. Práctica habitual: añadir H2 en un perfil test para los tests unitarios más simples, que así no dependen de tener Docker levantado.

La conexión: application-dev.yml

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/libreria_db
    username: libreria_user
    password: password123
```

`spring.datasource.*` es toda la información que Spring Boot necesita para conectar: la URL JDBC (protocolo, host, puerto y base de datos), usuario y contraseña. El pooling no lo configuras tú a mano: Spring Boot trae por defecto HikariCP, que se activa solo con tener el driver en el `pom.xml`.

La estructura: entidad Editorial

La estructura de la base de datos se declara sobre las propias clases Java, con anotaciones — el ORM que acabas de conocer, en código real:

```
@Entity
@Table(name = "editorial")
public class Editorial {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nombre;

    @OneToMany(mappedBy = "editorial", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Libro> libros;
}
```

La estructura: entidad Editorial (cont.)

- `@Entity + @Table(name = "editorial")`: esta clase se mapea contra la tabla editorial.
- `@Id + @GeneratedValue`: el identificador y cómo se genera (aquí, autoincremental, delegado en la propia base de datos).
- `@OneToMany(mappedBy = "editorial", ...)`: `mappedBy` indica que la clave foránea real vive en Libro (que llevará su propio `@ManyToOne` apuntando de vuelta a Editorial), no aquí.

La estructura: entidad Editorial (cont.)

- `cascade = CascadeType.ALL`: propaga las operaciones de Editorial a sus Libro — si guardas o borras una editorial, Hibernate aplica esa misma operación a todos sus libros automáticamente.
- `orphanRemoval = true`: si quitas un libro de la lista libros sin borrar la editorial entera, ese libro queda "huérfano" y Hibernate lo elimina de la base de datos por su cuenta.

`spring.jpa.hibernate.ddl-auto: update` hace que, al arrancar, Hibernate cree o actualice las tablas según esas anotaciones. En producción casi nunca se usa `update` ni `create-drop` — se prefieren migraciones controladas (Flyway, Liquibase).

Ideas clave

- El desfase objeto-relacional es la diferencia estructural entre cómo Java modela la información (objetos, referencias) y cómo lo hace SQL (tablas, claves foráneas) — no es un error, es una diferencia de naturaleza.
- Una herramienta ORM resuelve ese desfase; JPA es la especificación estándar de Java, Hibernate su implementación más usada, y Spring Data JPA añade los repositorios como capa de comodidad.
- JDBC es la API estándar de Java para hablar con bases de datos; un driver/conector la implementa para un gestor concreto.
- Un gestor embebido (H2, SQLite) corre dentro de tu proceso; uno independiente (PostgreSQL, MySQL) corre como servicio aparte, al que te conectas por red.

Ideas clave (cont.)

- El pooling de conexiones reutiliza conexiones ya abiertas en vez de crear una nueva cada vez — en Spring Boot lo gestiona HikariCP por defecto.
- Las relaciones entre entidades tienen multiplicidad: 1:1 (`@OneToOne`), 1:N (`@OneToMany`/`@ManyToOne`) y N:M (`@ManyToMany`, con tabla intermedia vía `@JoinTable`).
- En este curso el gestor es PostgreSQL independiente vía Docker; la conexión se configura en `application-dev.yml` (`spring.datasource.*`).
- La estructura se declara con anotaciones JPA sobre las propias clases Java; `mappedBy` señala el lado inverso, `cascade` propaga operaciones y `orphanRemoval` borra huérfanos. `ddl-auto` controla si Hibernate crea/actualiza las tablas.